



D Data
S Structures
A Algorithms

Data Structures & Algorithms

Today's Topic: Divide & Conquer



Dr. Si Thu Aung



in/drsithuaung/



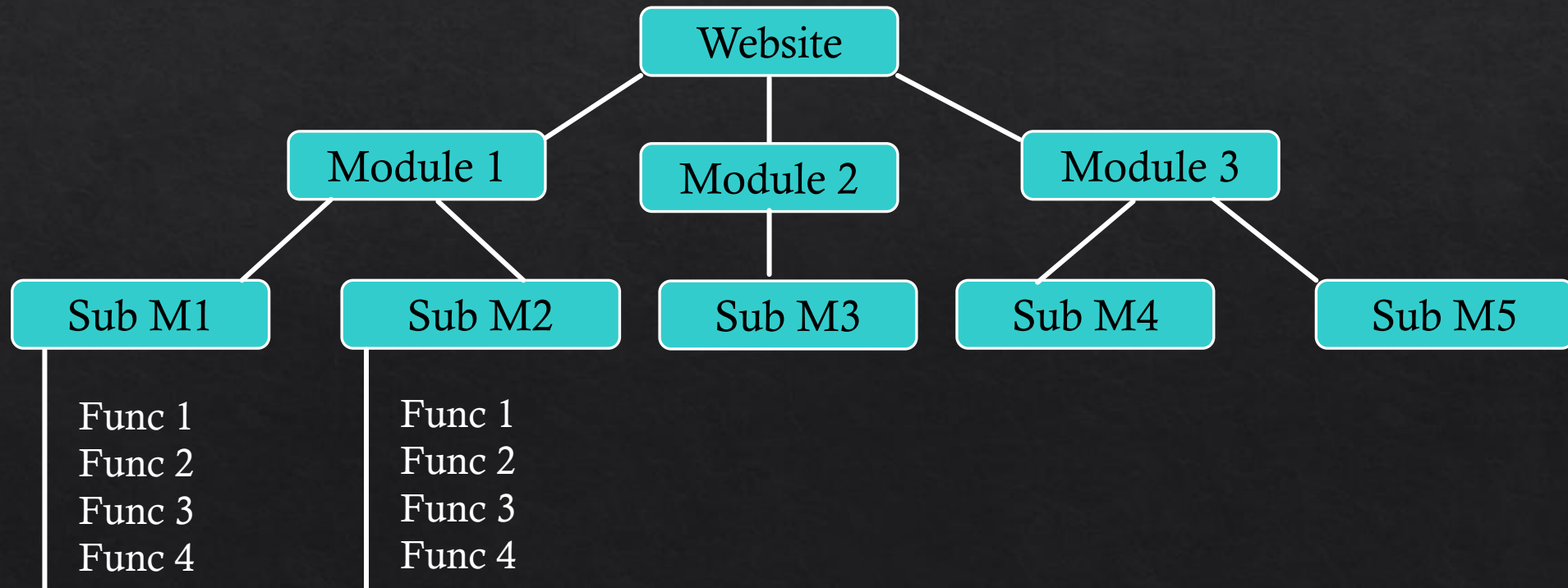
<https://sithuaung-ink.github.io/data-analyst-portfolio/>

5/12/2026

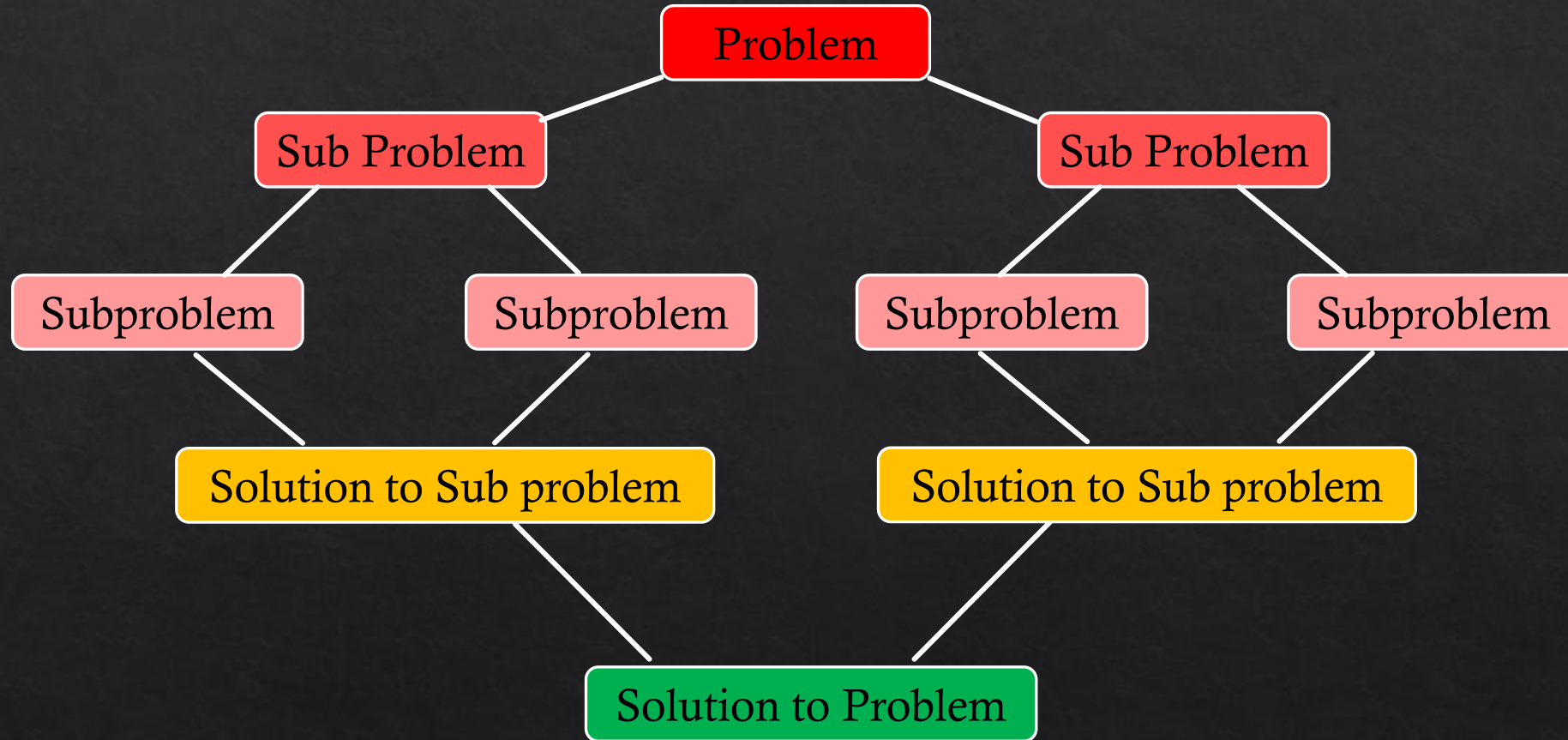
What is Divide and Conquer Algorithm?

Divide and conquer is an algorithm design paradigm that works by recursively breaking down a problem into subproblems of a similar type until these become simple enough to be solved directly. The solutions to the subproblems are then combined to give a solution to the original problem.

Example: Developing a website



What is Divide and Conquer Algorithm?



Property of Divide and Conquer Algorithm

Optimal Substructure:

If the overall solution to a problem can be constructed from the optimal solutions to its subproblems, then the problem has optimal substructure.

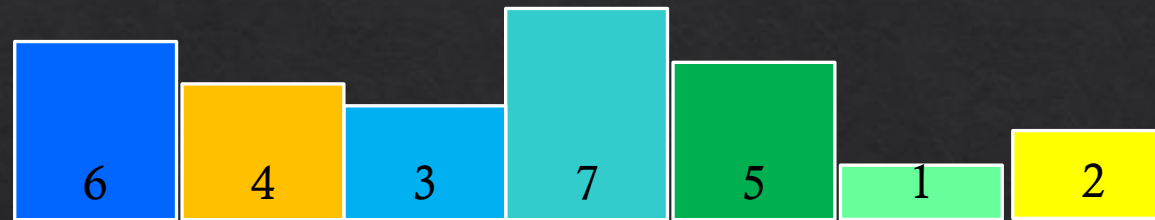
Example: $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

Why do we need it?

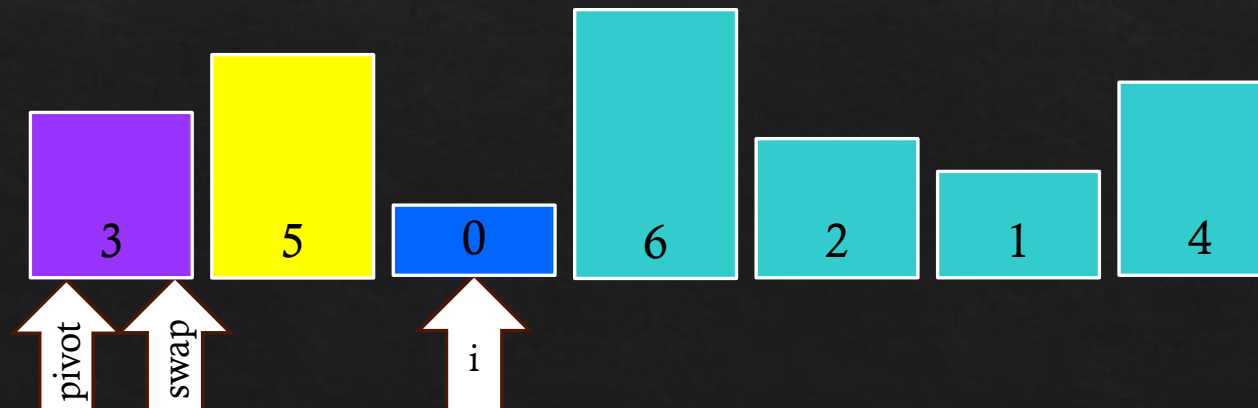
It is very effective when the problem has an optimal substructure property.

Common Divide and Conquer Algorithms

Merge Sort



Quick Sort

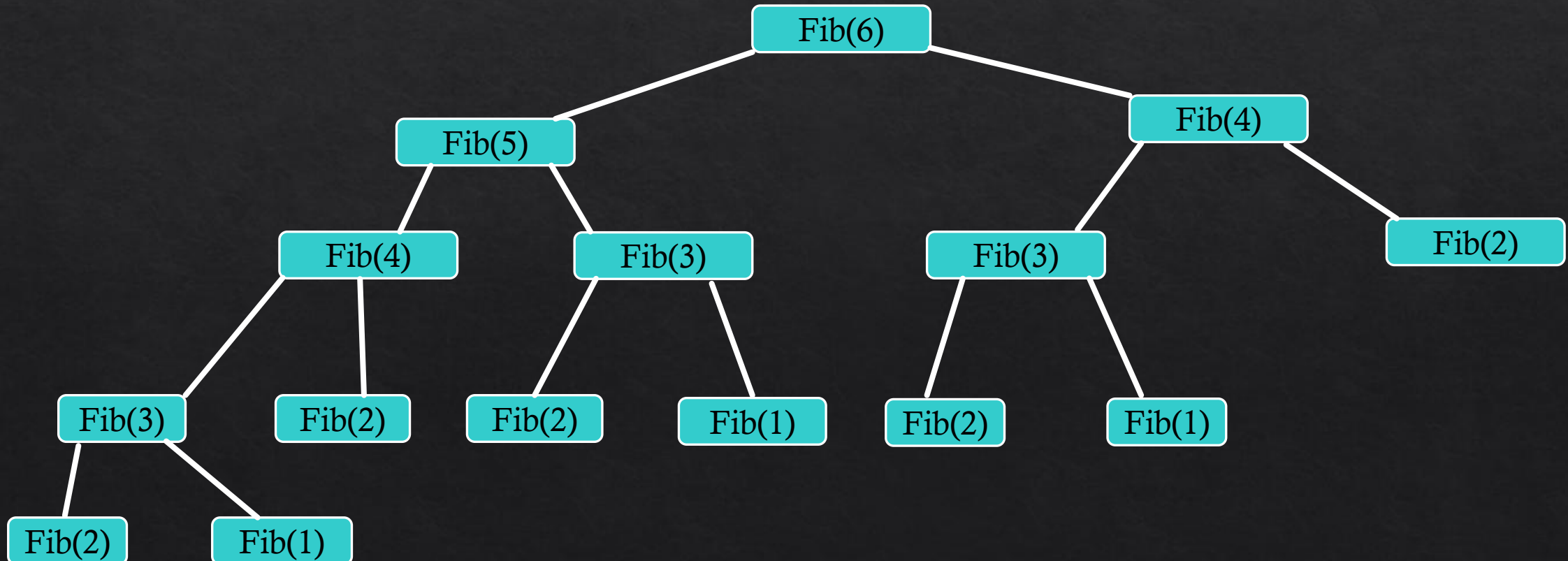


Fibonacci Series using Divide and Conquer Approach

The Fibonacci sequence is a sequence of numbers in which each number is the sum of the two preceding ones, and the sequence starts from 0 and 1.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

$$\text{Fibonacci}(N) = \text{Fibonacci}(N-1) + \text{Fibonacci}(N-2)$$



Fibonacci Series using Divide and Conquer Approach

The Fibonacci sequence is a sequence of numbers in which each number is the sum of the two preceding ones, and the sequence starts from 0 and 1.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

$$\text{Fibonacci}(N) = \text{Fibonacci}(N-1) + \text{Fibonacci}(N-2)$$

```
Fibonacci(N):  
    if n < 1 return error message  
    if n = 1 return 0  
    if n = 2 return 1  
    else:  
        return Fibonacci(N-1) + Fibonacci(N-2)
```

Divide and Conquer Algorithms



- Number Factor
- Zero One Knapsack
- Convert String

Number Factor

Problem Statement:

- Given N , find the number of ways to express N as a sum of 1, 3, and 4.

Example 1

- $N = 4$
- Number of ways = 4
- Explanation: There are 4 ways we can express $N \rightarrow \{4\}, \{3,1\}, \{1,3\}, \{1,1,1,1\}$

Example 2

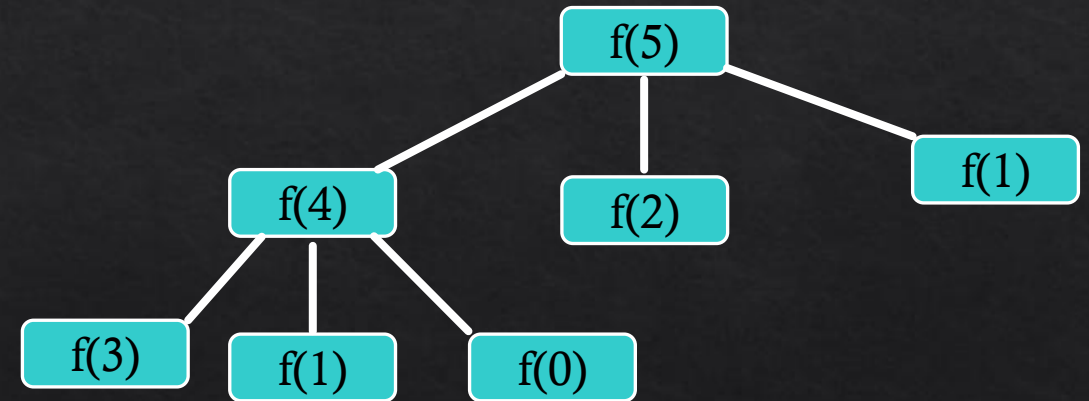
- $N = 5$
- Number of ways = 6
- Explanation: There are 6 ways we can express N .
 $\{4,1\}, \{1,4\}, \{3,1,1\}, \{1,3,1\}, \{1,1,3\}, \{1,1,1,1,1\}$

Number Factor

Example 2

- $N = 5$
- Number of ways = 6
- Explanation: There are 6 ways we can express N.
 $\{4,1\}, \{1,4\}, \{3,1,1\}, \{1,3,1\}, \{1,1,3\}, \{1,1,1,1,1\}$

$f(4)$	$\{4\}, \{3,1\}, \{1,3\}, \{1,1,1,1,1\}$	
$f(4)+1$	$\{4,1\}, \{3,1,1\}, \{1,3,1\}, \{1,1,1,1,1,1\}$	
$f(2)+3$	$\{1,1\}$	$\{1,1,3\}$
$f(1)+4$	$\{1\}$	$\{1,4\}$



Zero One Knapsack

Problem Statement:

- Given the weight and profits of N items
- Find the maximum profit within the given capacity of C
- Items cannot be broken

Example 1



Mango
Weight: 3
Profit: 31



Apple
Weight: 1
Profit: 26



Orange
Weight: 2
Profit: 17



Banana
Weight: 5
Profit: 72

Knapsack Capacity: 7

Answer Combinations

- $\text{Mango}(W:3, P:31) + \text{Apple}(W:1, P:26) + \text{Orange}(W:2, P:17) = W:6, P:74$
- $\text{Orange}(W:2, P:17) + \text{Banana}(W:5, P:72) = W:7, P: 89$
- $\text{Apple}(W:1, P:26) + \text{Banana}(W:5, P:72) = W:6, P:98$

Zero One Knapsack

Problem Statement:

- Given the weight and profits of N items
- Find the maximum profit within the given capacity of C
- Items cannot be broken

Example 1



Mango
Weight: 3
Profit: 31



Apple
Weight: 1
Profit: 26



Orange
Weight: 2
Profit: 17



Banana
Weight: 5
Profit: 72

Knapsack Capacity: 7

Subproblems:

- Option1 = $31 + f(2,3,4)$
- Option2 = $0 + f(2,3,4)$

Max(Option1, Option2)

Zero One Knapsack

Problem Statement:

- Given the weight and profits of N items
- Find the maximum profit within the given capacity of C
- Items cannot be broken

```
zoKnapsack(items, capacity, currentIndex):  
    if capacity <= 0 or currentIndex < 0 or currentIndex > len(profits):  
        return 0  
    elif currentItemWeight <= capacity:  
        Profit1 = currentItemProfit + zoKnapsack(items, capacity - currentItemWeight, nextItem)  
        Profit2 = zoKnapsack(items, capacity - currentItemWeight, nextItem)  
        return max(Profit1, Profit2)  
    else:  
        return 0
```


Convert String

Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert, or replace operations
- Find the minimum count of edit operations

Example

- | | | |
|--|---------|----------|
| <ul style="list-style-type: none">• S1 = "table"• S2 = "tgable" | Delete | $f(2,3)$ |
| <ul style="list-style-type: none">• S1 = "table"• S2 = "tble" | Insert | $f(3,2)$ |
| <ul style="list-style-type: none">• S1 = "table"• S2 = "tcble" | Replace | $f(3,3)$ |

Convert String

Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert, or replace operations
- Find the minimum count of edit operations

```
findMinOperation(s1, s2, index1, index2):  
    if index1 == len(s1):  
        return len(s2) - index2  
    if index2 == len(s2):  
        return len(s1) - index1  
    if s1[index1] == s2[index2]:  
        return findMinOperation(s1, s2, index1+1, index2+1)  
    else:  
        deleteOp = 1 + findMinOperation(s1, s2, index1, index2+1)  
        insertOp = 1 + findMinOperation(s1, s2, index1+1, index2)  
        replaceOp = 1 + findMinOperation(s1, s2, index1+1, index2+1)  
        return min(deleteOp, insertOp, replaceOp)
```